

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №2
по курсу «Операционные системы»

Выполнил: А. В. Козлов
Группа: М8О-207БВ-24
Преподаватель: Е. С. Миронов

Москва, 2025

Условие

Цель работы:

приобретение практических навыков в:

- Управление потоками в ОС
- Обеспечение синхронизации между потоками

Задание: Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработке использовать стандартные средства создания потоков операционной системы (Windows/Unix). Ограничение максимального количества потоков, работающих в один момент времени, должно быть задано ключом запуска вашей программы. Так же необходимо уметь продемонстрировать количество потоков, используемое вашей программой с помощью стандартных средств операционной системы. В отчете привести исследование зависимости ускорения и эффективности алгоритма от входных данных и количества потоков. Получившиеся результаты необходимо объяснить.

Произвести перемножение 2-ух матриц, содержащих комплексные числа

Вариант: 6

Метод решения

Для умножения матриц с комплексными числами используется стандартный алгоритм, где каждый элемент результата получается перемножением и сложением элементов из строки первой матрицы и столбца второй. Умножение комплексных чисел выполняется по правилу: действительные и мнимые части перемножаются особым образом с учетом формулы для комплексных чисел.

Чтобы ускорить вычисления для больших матриц, применяется многопоточность. Строки результирующей матрицы делятся между потоками, и каждый поток независимо вычисляет свою часть. Это позволяет задействовать несколько ядер процессора одновременно.

Перед началом вычислений проверяется, что количество столбцов первой матрицы совпадает с количеством строк второй — это необходимое условие для умножения матриц. Комплексные числа хранятся в виде пар действительная и мнимая часть, для которых определены операции сложения и умножения.

Описание программы

Файл `matrix.hpp` содержит объявления основных структур данных и функций для работы с матрицами. Определены типы данных для комплексных чисел и матриц, а также структура для передачи данных в потоки. Реализованы функции создания матрицы, умножения матриц и вывода результата.

Файл `thread.hpp` предоставляет интерфейс для работы с потоками через класс `Thread`. Реализованы основные операции управления потоками: создание, ожидание завершения и освобождение ресурсов. Класс инкапсулирует работу с системными вызовами `pthread`.

Файл `matrix.cpp` содержит реализацию алгоритмов работы с матрицами. Функция умножения матриц поддерживает как однопоточное выполнение, так и многопоточную обработку с распределением строк между потоками. Каждый поток независимо вычисляет свой блок строк результирующей матрицы.

Файл `thread.cpp` реализует механизм работы с потоками. Используются системные вызовы `pthread_create` для создания потоков, `pthread_join` для ожидания завершения и `pthread_detach`

для освобождения ресурсов. Класс Thread управляет жизненным циклом потоков и обеспечивает безопасное освобождение ресурсов.

Файл main.cpp является точкой входа в программу. Получает от пользователя количество потоков и размеры матриц, создает матрицы с тестовыми данными, выполняет умножение с замером времени и выводит результат работы.

Основные типы данных:

Complex - комплексное число

Matrix - двумерная матрица комплексных чисел

MatrixThreadData - структура для передачи данных в потоки

Используемые системные вызовы:

pthread_create - создание потока

pthread_join - ожидание завершения потока

pthread_detach - освобождение ресурсов потока

Результаты

При запуске программы с нужным количеством потоков и вводе размеров матриц А и В программа производит умножение, замеряя время выполнения алгоритма, и в конце выводит время выполнения и, при надобности, саму матрицу

Пример работы:

```
./main 4
```

Введите размеры матрицы А

```
4 4
```

введите размеры матрицы В

```
4 4
```

Вывод в консоль:

```
993ms
```

При запуске программы по разным размерам матриц и разным количеством потоков можно проследить зависимость скорости выполнения от размеров матриц и количеством потоков

Анализ производительности матричного умножения

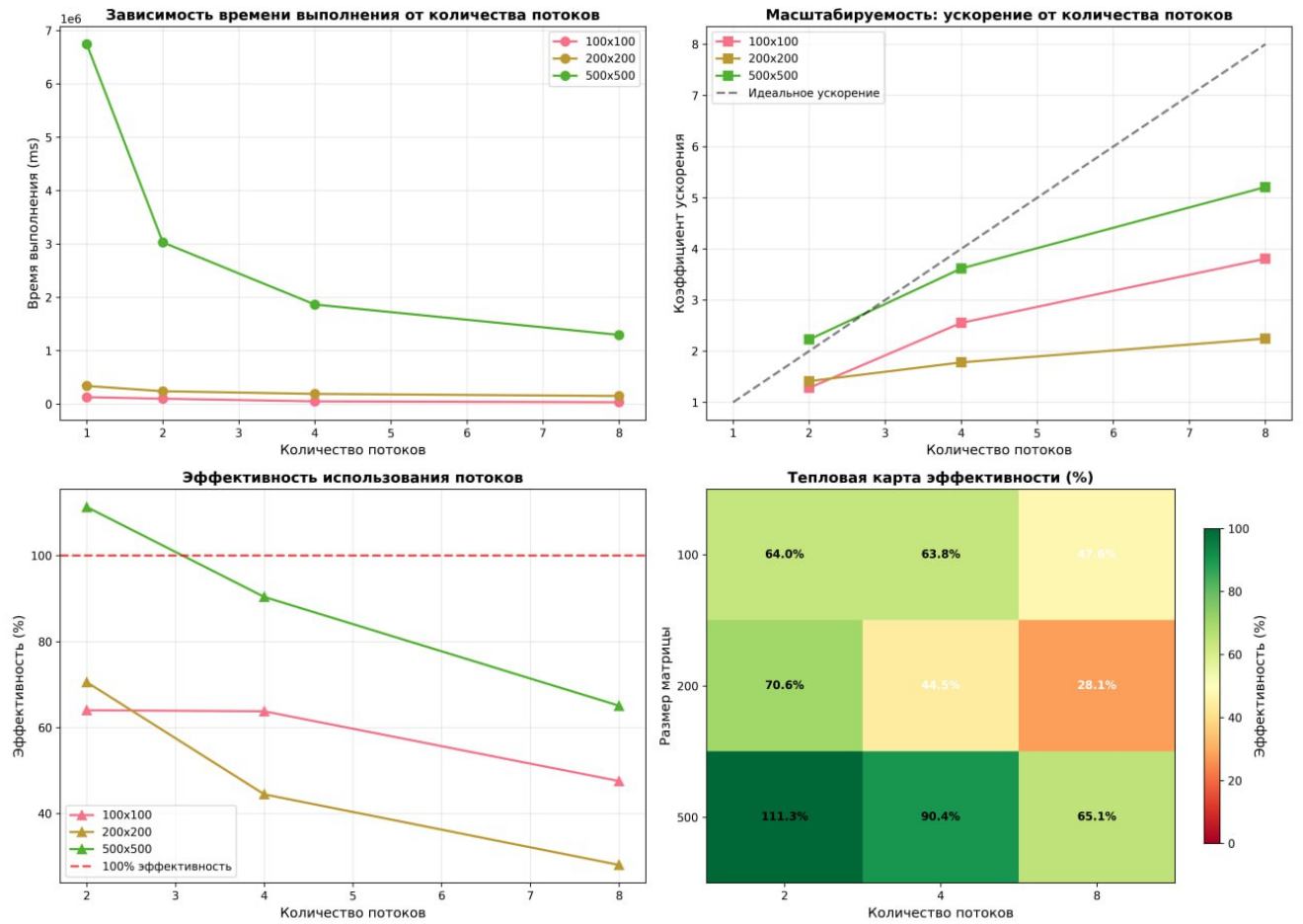


Рис.1 Графики анализа производительности

Выводы

Разработанная программа успешно реализует параллельное умножение матриц с комплексными числами. Многопоточная обработка обеспечивает ускорение вычислений, особенно заметное на матрицах крупного размера.

Архитектура демонстрирует стабильную работу с различными размерами матриц и количеством потоков. Наибольшая эффективность достигается при обработке матриц от 500×500 элементов.

Программа корректно управляет ресурсами потоков и обеспечивает проверку входных данных, что гарантирует надежность выполнения операций.

Исходная программа

```
1  #pragma once
2
3  #include <pthread.h>
4
5  namespace thread{
6  struct ThreadData;
7  using ThreadFunc = typedef(void*(void*))*;
8  class Thread {
9  public:
10
11     Thread(ThreadFunc func);
12
13     Thread(const Thread&) = delete;
14
15     Thread& operator=(const Thread&) = delete;
16
17     Thread(Thread&& other) noexcept;
18
19     Thread& operator=(Thread&& other) noexcept;
20
21     void Join();
22
23     void Detach();
24
25     void Run(void *);
26
27     ~Thread();
28 private:
29     ThreadFunc func_;
30     ThreadData* data_;
31 };
32 } // namespace thread
```

Листинг 1: *Интерфейс потоков*

```
1  #include <stdexcept>
2  #include <iostream>
3  #include <cstring>
4
5  #include "thread.hpp"
6
7  namespace thread{
8
9  struct ThreadData
10 {
11     pthread_t thread = 0;
12     bool is_running = false;
13 };
14
15 Thread::Thread(ThreadFunc func) : func_(func) {
16     data_ = new ThreadData();
17 }
18
19 void Thread::Join() {
20     int code = pthread_join(data_->thread, NULL);
```

```

21     if(code != 0) {
22         throw std::runtime_error("thread join error");
23     }
24 }
25
26 void Thread::Detach() {
27     int code = pthread_detach(data_>thread);
28     if(code != 0) {
29         throw std::runtime_error("thread detach error");
30     }
31     data_>is_running = false;
32 }
33
34 Thread::Thread(Thread&& other) noexcept : func_(other.func_), data_(other.data_) {
35     other.data_ = nullptr;
36     other.func_ = nullptr;
37 }
38
39
40 Thread& Thread::operator=(Thread&& other) noexcept {
41     if (this != &other) {
42         if (data_) {
43             delete data_;
44         }
45         func_ = other.func_;
46         data_ = other.data_;
47         other.func_ = nullptr;
48         other.data_ = nullptr;
49     }
50     return *this;
51 }
52
53 void Thread::Run(void *data) {
54     int code = pthread_create(&(data_>thread), NULL, func_, data);
55     if(code != 0) {
56         throw std::runtime_error("thread create error");
57     }
58 }
59
60 Thread::~~Thread() {
61     if (data_) {
62         delete data_;
63         data_ = nullptr;
64     }
65 }
66 } // namespace thread

```

Листинг 2: *Реализация потоков*

```

1  #pragma once
2
3  #include <vector>
4  #include <complex>
5
6  #include "thread.hpp"
7
8  using Complex = std::complex<double>;

```

```

9  using Matrix = std::vector<std::vector<Complex>>>;
10
11  struct MatrixThreadData {
12      const Matrix* a;
13      const Matrix* b;
14      Matrix* result;
15      int start_row;
16      int end_row;
17  };
18
19  Matrix create_matrix(int rows, int cols);
20
21  void* multiply_thread(void* arg);
22
23  Matrix multiply_matrices(const Matrix& a, const Matrix& b, int num_threads = 2);
24
25  void print_matrix(const Matrix& m);

```

Листинг 3: *Интерфейс матричных функций*

```

1  #include <iostream>
2  #include <iomanip>
3  #include <vector>
4
5  #include "matrix.hpp"
6
7  Matrix create_matrix(int rows, int columns) {
8      Matrix m(rows, std::vector<Complex>(columns));
9      for (int i = 0; i < rows; i++) {
10         for (int j = 0; j < columns; j++) {
11             m[i][j] = Complex(i + j, i - j);
12         }
13     }
14     return m;
15 }
16
17 void* multiply_thread(void* arg) {
18     MatrixThreadData* data = (MatrixThreadData*)arg;
19     const Matrix& a = *(data->a);
20     const Matrix& b = *(data->b);
21     Matrix& result = *(data->result);
22
23     int cols_b = b[0].size();
24     int cols_a = a[0].size();
25
26     for (int i = data->start_row; i < data->end_row; i++) {
27         for (int j = 0; j < cols_b; j++) {
28             result[i][j] = 0;
29             for (int k = 0; k < cols_a; k++) {
30                 result[i][j] += a[i][k] * b[k][j];
31             }
32         }
33     }
34
35     return nullptr;
36 }
37

```



```

38 Matrix multiply_matrices(const Matrix& a, const Matrix& b, int num_threads) {
39     int rows_a = a.size();
40     int cols_b = b[0].size();
41
42     if (a.empty() || b.empty() || a[0].size() != b.size()) {
43         throw std::runtime_error("invalid matrix size");
44     }
45
46     Matrix result(rows_a, std::vector<Complex>(cols_b, 0));
47
48     if (num_threads == 1 || rows_a < num_threads) {
49         MatrixThreadData data{&a, &b, &result, 0, rows_a};
50         multiply_thread(&data);
51         return result;
52     }
53
54     num_threads = std::min(num_threads, rows_a);
55
56     std::vector<thread::Thread> threads;
57     std::vector<MatrixThreadData> thread_data(num_threads);
58     int rows_per_thread = rows_a / num_threads;
59     int current_row = 0;
60
61     for (int i = 0; i < num_threads; i++) {
62         int start = current_row;
63         int end = (i == num_threads - 1) ? rows_a : current_row + rows_per_thread;
64         thread_data[i] = {&a, &b, &result, start, end};
65         threads.emplace_back(multiply_thread);
66         threads.back().Run(&thread_data[i]);
67         current_row = end;
68     }
69
70     for (int i = 0; i < num_threads; i++) {
71         threads[i].Join();
72     }
73
74     return result;
75 }
76
77 void print_matrix(const Matrix& m) {
78     for (const auto& row : m) {
79         for (const auto& elem : row) {
80             std::cout << std::setw(8) << std::fixed << std::setprecision(2)
81                 << elem.real() << (elem.imag() >= 0 ? "+" : "")
82                 << elem.imag() << "i ";
83         }
84         std::cout << "\n";
85     }
86     std::cout << std::endl;
87 }

```

Листинг 4: *Реализация матричных функций*

```

1 #include <iostream>
2 #include <chrono>
3
4 #include "matrix.hpp"

```

```

5  #include "thread.hpp"
6
7  int main(int argc, char* argv[]) {
8      if(argc < 2) {
9          std::cerr << "not enough args, needed threads";
10         return 1;
11     }
12     int threads_count = std::stoi(argv[1]);
13     int rows_a;
14     int cols_a;
15     int rows_b;
16     int cols_b;
17
18     std::cout << "    \n";
19     std::cin >> rows_a >> cols_a;
20     std::cout << "    \n";
21     std::cin >> rows_b >> cols_b;
22     Matrix A = create_matrix(rows_a, cols_a);
23     Matrix B = create_matrix(rows_b, cols_b);
24
25     auto begin = std::chrono::steady_clock::now();
26     Matrix result = multiply_matrices(A, B, threads_count);
27     auto end = std::chrono::steady_clock::now();
28     auto duration = std::chrono::duration_cast<std::chrono::microseconds>(end - begin)
29         .count();
30     std::cout << duration << "ms\n";
31 }

```

Листинг 5: *Основной файл программы*

```

execve("./main",["./main","4"],0x7fff76f52f38 /* 48 vars */) = 0
brk(NULL) = 0x637dd3546000
mmap(NULL,8192,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_ANONYMOUS,-1,0)
= 0x73609f433000
access("/etc/ld.so.preload",R_OK) = -1 ENOENT (No such file or
directory)
openat(AT_FDCWD,"/etc/ld.so.cache",O_RDONLY|O_CLOEXEC) = 3
fstat(3,{st_mode=S_IFREG|0644,st_size=59571,...}) = 0
mmap(NULL,59571,PROT_READ,MAP_PRIVATE,3,0) = 0x73609f424000
close(3) = 0

openat(AT_FDCWD,"/lib/x86_64-linux-gnu/libstdc++.so.6",O_RDONLY|O_CLOEXEC)
= 3

read(3,"\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0"... ,832)
= 832
fstat(3,{st_mode=S_IFREG|0644,st_size=2592224,...}) = 0
mmap(NULL,2609472,PROT_READ,MAP_PRIVATE|MAP_DENYWRITE,3,0) =
0x73609f000000

mmap(0x73609f09d000,1343488,PROT_READ|PROT_EXEC,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3
= 0x73609f09d000

```

```

mmap(0x73609f1e5000,552960,PROT_READ,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,0x1e5000)
= 0x73609f1e5000

mmap(0x73609f26c000,57344,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,
= 0x73609f26c000

mmap(0x73609f27a000,12608,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS,-1
= 0x73609f27a000
    close(3)                                = 0

openat(AT_FDCWD,"/lib/x86_64-linux-gnu/libgcc_s.so.1",O_RDONLY|O_CLOEXEC)
= 3

read(3,"\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0"... ,832)
= 832
    fstat(3,{st_mode=S_IFREG|0644,st_size=183024,...}) = 0
    mmap(NULL,185256,PROT_READ,MAP_PRIVATE|MAP_DENYWRITE,3,0) =
0x73609f3f6000

mmap(0x73609f3fa000,147456,PROT_READ|PROT_EXEC,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,
= 0x73609f3fa000

mmap(0x73609f41e000,16384,PROT_READ,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,0x28000)
= 0x73609f41e000

mmap(0x73609f422000,8192,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,0
= 0x73609f422000
    close(3)                                = 0

openat(AT_FDCWD,"/lib/x86_64-linux-gnu/libc.so.6",O_RDONLY|O_CLOEXEC) = 3

read(3,"\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"... ,832)
= 832

pread64(3,"\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"... ,784,64)
= 784
    fstat(3,{st_mode=S_IFREG|0755,st_size=2125328,...}) = 0

pread64(3,"\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"... ,784,64)
= 784
    mmap(NULL,2170256,PROT_READ,MAP_PRIVATE|MAP_DENYWRITE,3,0) =
0x73609ec00000

mmap(0x73609ec28000,1605632,PROT_READ|PROT_EXEC,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3
= 0x73609ec28000

mmap(0x73609edb0000,323584,PROT_READ,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,0x1b0000)

```

```
= 0x73609edb0000
```

```
mmap(0x73609edff000,24576,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,  
= 0x73609edff000
```

```
mmap(0x73609ee05000,52624,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS,-1,  
= 0x73609ee05000  
close(3) = 0
```

```
openat(AT_FDCWD,"/lib/x86_64-linux-gnu/libm.so.6",O_RDONLY|O_CLOEXEC) = 3
```

```
read(3,"\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0"... ,832)  
= 832
```

```
fstat(3,{st_mode=S_IFREG|0644,st_size=952616,...}) = 0  
mmap(NULL,950296,PROT_READ,MAP_PRIVATE|MAP_DENYWRITE,3,0) =  
0x73609f30d000
```

```
mmap(0x73609f31d000,520192,PROT_READ|PROT_EXEC,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,  
= 0x73609f31d000
```

```
mmap(0x73609f39c000,360448,PROT_READ,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,0x8f000)  
= 0x73609f39c000
```

```
mmap(0x73609f3f4000,8192,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,0  
= 0x73609f3f4000
```

```
close(3) = 0  
mmap(NULL,8192,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_ANONYMOUS,-1,0)  
= 0x73609f30b000
```

```
mmap(NULL,12288,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_ANONYMOUS,-1,0) =  
0x73609f308000
```

```
arch_prctl(ARCH_SET_FS,0x73609f308740) = 0  
set_tid_address(0x73609f308a10) = 8023  
set_robust_list(0x73609f308a20,24) = 0  
rseq(0x73609f309060,0x20,0,0x53053053) = 0  
mprotect(0x73609edff000,16384,PROT_READ) = 0  
mprotect(0x73609f3f4000,4096,PROT_READ) = 0  
mprotect(0x73609f422000,4096,PROT_READ) = 0  
mprotect(0x73609f26c000,45056,PROT_READ) = 0  
mprotect(0x637dc3ed8000,4096,PROT_READ) = 0  
mprotect(0x73609f471000,8192,PROT_READ) = 0
```

```
prlimit64(0,RLIMIT_STACK,NULL,{rlim_cur=8192*1024,rlim_max=RLIM64_INFINITY})  
= 0
```

```
munmap(0x73609f424000,59571) = 0  
futex(0x73609f27a7bc,FUTEX_WAKE_PRIVATE,2147483647) = 0  
getrandom("\x1e\x77\x89\x3b\xf2\x48\xbb\xb1",8,GRND_NONBLOCK) = 8  
brk(NULL) = 0x637dd3546000
```

```

brk(0x637dd3567000) = 0x637dd3567000
fstat(1,{st_mode=S_IFCHR|0620,st_rdev=makedev(0x88,0x3),...}) = 0
write(1,"\320\222\320\262\320\265\320\264\320\270\321\202\320\265
\321\200\320\260\320\267\320\274\320\265\321\200\321\213 \320\274"... ,48)
= 48
fstat(0,{st_mode=S_IFCHR|0620,st_rdev=makedev(0x88,0x3),...}) = 0
read(0,"4\n",1024) = 2
read(0,"4\n",1024) = 2
write(1,"\320\262\320\262\320\265\320\264\320\270\321\202\320\265
\321\200\320\260\320\267\320\274\320\265\321\200\321\213 \320\274"... ,48)
= 48
read(0,"4\n",1024) = 2
read(0,"4\n",1024) = 2

rt_sigaction(SIGRT_1,{sa_handler=0x73609ec99530,sa_mask=[],sa_flags=SA_RESTORER|SA_ON
= 0
rt_sigprocmask(SIG_UNBLOCK,[RTMIN RT_1],NULL,8) = 0

mmap(NULL,8392704,PROT_NONE,MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK,-1,0) =
0x73609e3ff000
mprotect(0x73609e400000,8388608,PROT_READ|PROT_WRITE) = 0
rt_sigprocmask(SIG_BLOCK,~[],[],8) = 0

clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|
=>{parent_tid=[8024]},88) = 8024
rt_sigprocmask(SIG_SETMASK,[],NULL,8) = 0

mmap(NULL,8392704,PROT_NONE,MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK,-1,0) =
0x73609dbfe000
mprotect(0x73609dbff000,8388608,PROT_READ|PROT_WRITE) = 0
rt_sigprocmask(SIG_BLOCK,~[],[],8) = 0

clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|
=>{parent_tid=[8025]},88) = 8025
rt_sigprocmask(SIG_SETMASK,[],NULL,8) = 0

mmap(NULL,8392704,PROT_NONE,MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK,-1,0) =
0x73609d3fd000
mprotect(0x73609d3fe000,8388608,PROT_READ|PROT_WRITE) = 0
rt_sigprocmask(SIG_BLOCK,~[],[],8) = 0

clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|
=>{parent_tid=[8026]},88) = 8026
rt_sigprocmask(SIG_SETMASK,[],NULL,8) = 0

mmap(NULL,8392704,PROT_NONE,MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK,-1,0) =
0x73609cbfc000
mprotect(0x73609cbfd000,8388608,PROT_READ|PROT_WRITE) = 0

```

```

rt_sigprocmask(SIG_BLOCK,~[],[],8)    = 0

clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|
=>{parent_tid=[8027]}},88) = 8027
    rt_sigprocmask(SIG_SETMASK,[],NULL,8) = 0
    write(1,"5986ms\n",7)                = 7
    lseek(0,-1,SEEK_CUR)                   = -1 ESPIPE (Illegal seek)
    exit_group(0)                          = ?
+++ exited with 0 +++

```

Листинг 6: *Strace логи*